Lab2: EEG classification

312554029 陳映璇

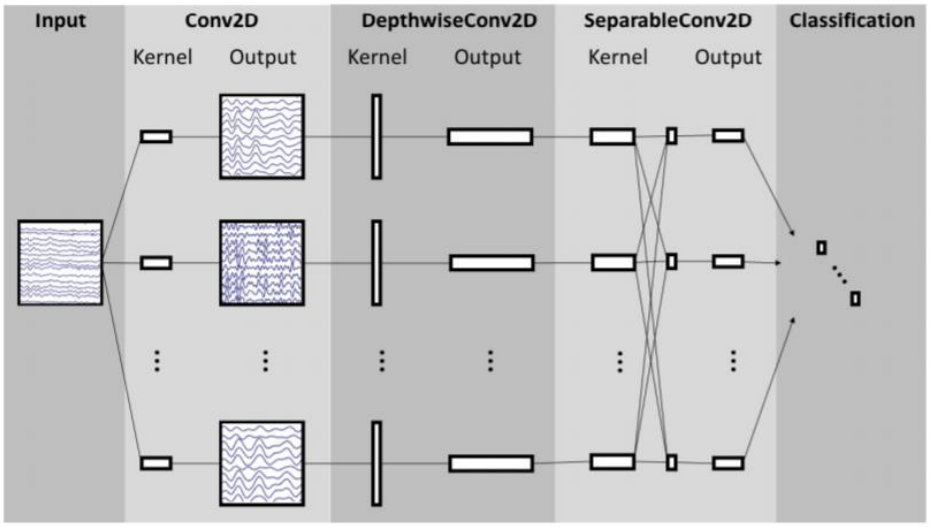
1. Introduction

In this lab, I implemented two simple EEG classification networks, EEGNet and DeepConvNet, using three kinds of activation functions including ReLU, Leaky ReLU, and ELU. I adjusted different hyper-parameters several times in my experiments, and ultimately obtained the highest accuracy for each network with different activation functions.

2. Experiment setups

A. Details of the two models

i. EEGNet




```
class EEGNet(nn.Module):
    def __init__(self, act = cfg.act):
        super().__init__()
```

- First, I define a class called *EEGNet* and make it inherits all the functionalities of *nn.Module*.

```
self.firstconv = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size = (1,51), stride = (1,1), padding = (0,25), bias = False),
    nn.BatchNorm2d(16, eps = 1e-5, momentum = 0.1, affine = True, track_running_stats = True)
)
```

- The purpose of this convolutional layer is to extract some low-

level features from the input data. The kernel has a height of 1, which means it focuses only on temporal features, and a width of 51 to cover a longer time window. Batch normalization is also used here to accelerate training and reduce gradient vanishing problems. The resulting output will have 16 feature maps, which will be further processed by subsequent layers in the EEGNet architecture.

```
self.depthwiseConv = nn.Sequential(
    nn.Conv2d(16, 32, kernel_size = (2,1), stride=(1,1), groups = 16, bias = False),
    nn.BatchNorm2d(32, eps = 1e-5, momentum = 0.1, affine = True, track_running_stats = True),
    self.activation(act),
    nn.AvgPool2d(kernel_size = (1,4), stride = (1,4), padding = 0),
    nn.Dropout(p = 0.25)
)
```

- This part performs a **Depthwise convolution** that aims to reduce computational complexity by performing group convolutions on 16 channels independently. It uses a kernel size of (2,1) to perform separate convolutions on each channel, extracting features in the channel direction.

```
self.separableConv = nn.Sequential(
    nn.Conv2d(32, 32, kernel_size = (1,15), stride = (1,1), padding = (0,7), bias = False),
    nn.BatchNorm2d(32, eps = 1e-5, momentum = 0.1, affine = True, track_running_stats = True),
    self.activation(act),
    nn.AvgPool2d(kernel_size = (1,8), stride = (1,8), padding = 0),
    nn.Dropout(p = 0.25)
)
```

- This part performs a **Pointwise convolution**, which uses 1x1 convolutional filters to mix the channels produced by the depthwise convolution. The purpose here is to extract features in both temporal and channel directions.
- The above two layers implement **Depthwise Separable Convolution**, which aims to extract relevant features from the input data and reduce its spatial dimensions while promoting generalization and preventing overfitting during training.

```
self.classify = nn.Sequential(
    nn.Flatten(),
    nn.Linear(in_features = 736, out_features = 2, bias = True)
)
```

- This part is the output layer of the EEGNet model, responsible for producing the final classification logits.
- *nn.Flatten()* is responsible for flattening the input tensor x into a

1D vector. This is necessary because the preceding layers (*self.separableConv*) output a 4D tensor, but we need to transform it into a 1D tensor to feed it into a fully connected layer for classification.

ii. DeepConvNet

Layer	# filters	size	# params	Activation	Options
Input		(C, T)			
Reshape		(1, C, T)			
Conv2D	25	(1, 5)	150	Linear	mode = valid, max norm = 2
Conv2D	25	(C, 1)	25 * 25 * C + 25	Linear	mode = valid, max norm = 2
BatchNorm			2 * 25		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	50	(1, 5)	25 * 50 * C + 50	Linear	mode = valid, max norm = 2
BatchNorm			2 * 50		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	100	(1, 5)	50 * 100 * C + 100	Linear	mode = valid, max norm = 2
BatchNorm			2 * 100		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	200	(1, 5)	100 * 200 * C + 200	Linear	mode = valid, max norm = 2
BatchNorm			2 * 200		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Flatten					
Dense	N			softmax	max norm = 0.5

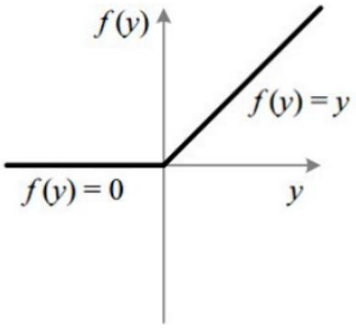
- *DeepConvNet* consists of four successive convolutional blocks with increasing numbers of filters and decreasing spatial dimensions (due to max pooling).

B. Explain the activation function (ReLU, Leaky ReLU, ELU)

i. **Rectified Linear Unit (ReLU)**

$$Re\,LU\,(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

$$\mathrm{Re}\,LU'\,(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

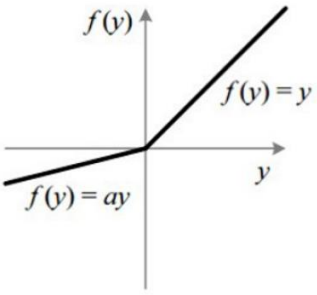


- Advantages:
  - Efficient computation and easy implementation.
  - Helps address the vanishing gradient problem.
  - Induces sparsity, activating only a subset of neurons.
- Disadvantages:
  - Prone to the "dying ReLU" problem, where neurons can get stuck during training and never activate again.
  - Lacks negative activation, making it unsuitable for tasks where negative values are significant.
  - Not suitable for all types of data, as it can cause the "exploding gradient" problem with extremely large inputs.

ii. Leaky Rectified Linear Unit (Leaky ReLU)

$$LeakyReLU\ (x) = \begin{cases} 0.1 * \mathrm{x}, & x \leq 0 \\ x, & x > 0 \end{cases}$$

$$LeakyReLU'\ (x) = \begin{cases} 0.1, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

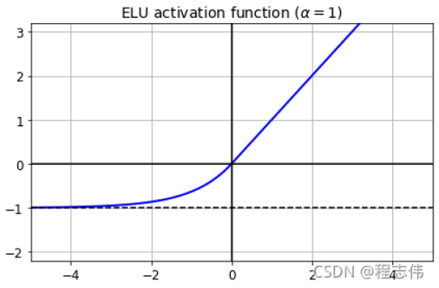


- Advantages:
  - Solves the "dying ReLU" problem by preventing neurons from getting stuck at zero during training.
  - Helps address the vanishing gradient problem.
  - Efficient computation and easy implementation.
- Disadvantages:
  - The slope parameter needs to be tuned, which can be an additional hyperparameter to optimize.
  - Lacks the sparsity-inducing property of ReLU.
  - May introduce some noise in the output for negative inputs due to the small slope.

iii. Exponential Linear Unit (ELU)

$$ELU\ (x) = \begin{cases} a * (e^x - 1), x \leq 0 \\ x, \qquad\qquad x > 0 \end{cases}$$

$$ELU(x) = \begin{cases} ELU(x) + a, x \leq 0 \\ 1, \qquad\qquad x > 0 \end{cases}$$



- Advantages:
  - Solves the "dying ReLU" problem by preventing neurons from getting stuck at zero during training.
  - Helps address the vanishing gradient problem better than ReLU and Leaky ReLU.
  - Smooth and continuous, which helps with optimization during training.
- Disadvantages:
  - The slope parameter needs to be tuned, which can be an additional hyperparameter to optimize.
  - Computationally more expensive than ReLU and its variants due to the exponential calculation.
  - Introduces saturation for extremely negative inputs, which may slow down learning in some cases.

3. Experimental results

A. The highest testing accuracy

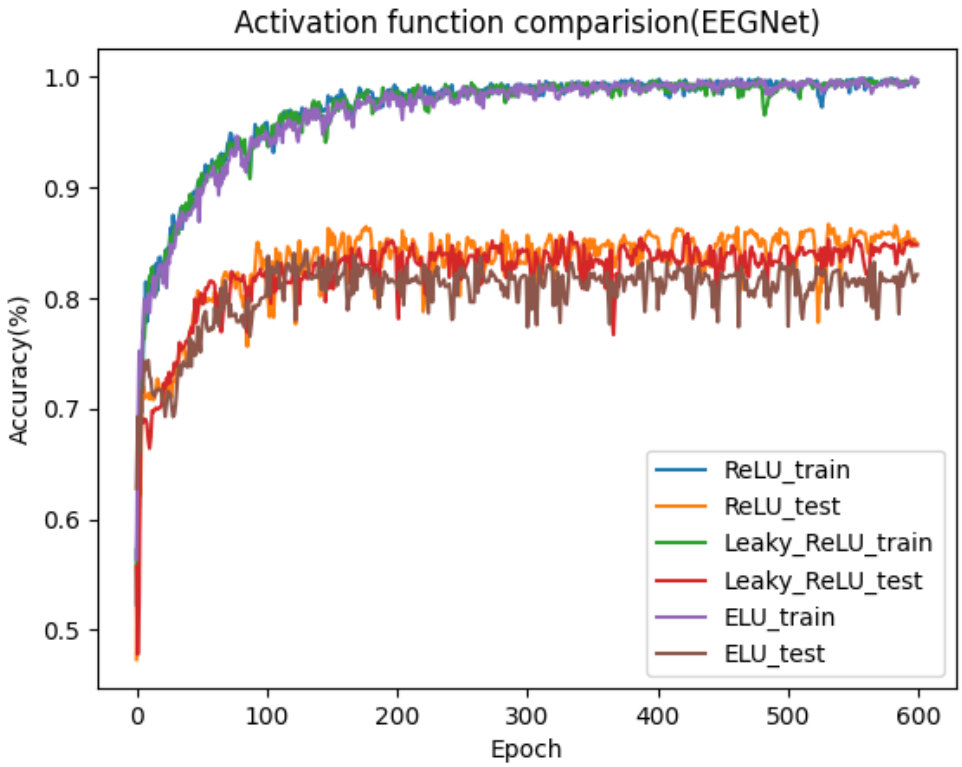
Activation	ReLU	Leaky ReLU	ELU
EEGNet	88.26%	88.20%	85.91%
DeepConvNet	83.11%	83.56%	82.77%

epoch	batch size	T0	lr	min_lr	EEG_ReLU	EEG_Leaky	EEG_ELU		DeepConv_ReLU	DeepConv_Leaky	DeepConv_ELU
150	64	20	1.00E-02	8.00E-09	80.95%	81.68%	79.15%		77.81%	78.14%	77.64%
300	64	20	1.00E-02	8.00E-09	81.78%	82.31%	79.15%		77.81%	78.73%	78.87%
600	64	20	1.00E-02	8.00E-09	82.90%	82.60%	79.48%		79.02%	78.73%	78.87%
1000	64	20	1.00E-02	8.00E-09	82.90%	82.90%	78.68%		79.41%	80.42%	78.87%
150	128	20	1.00E-02	8.00E-09	81.77%	81.82%	81.31%		77.03%	75.89%	78.78%
300	128	20	1.00E-02	8.00E-09	82.84%	82.39%	82.01%		78.18%	76.84%	79.64%
600	128	20	1.00E-02	8.00E-09	83.42%	83.62%	80.59%		75.73%	78.34%	80.31%
1000	128	20	1.00E-02	8.00E-09	82.96%	82.79%	81.71%		79.71%	78.99%	78.70%
150	512	20	1.00E-02	8.00E-09	86.76%	85.51%	83.87%		78.56%	78.83%	82.25%
300	512	20	1.00E-02	8.00E-09	87.34%	86.75%	84.79%		79.05%	80.14%	82.70%
600	512	20	1.00E-02	8.00E-09	87.99%	87.93%	84.92%		81.25%	81.01%	82.77%
1000	512	20	1.00E-02	8.00E-09	88.00%	88.06%	85.91%		83.11%	82.19%	82.77%
150	512	20	1.02E-02	8.00E-08	87.23%	87.03%	82.80%		77.33%	78.23%	76.07%
300	512	20	1.02E-02	8.00E-08	88.26%	87.81%	84.79%		78.51%	80.01%	78.62%
600	512	20	1.02E-02	8.00E-08	88.26%	88.20%	84.79%		79.30%	80.87%	80.72%
1000	512	20	1.02E-02	8.00E-08	88.26%	88.20%	84.79%		79.35%	83.56%	80.72%

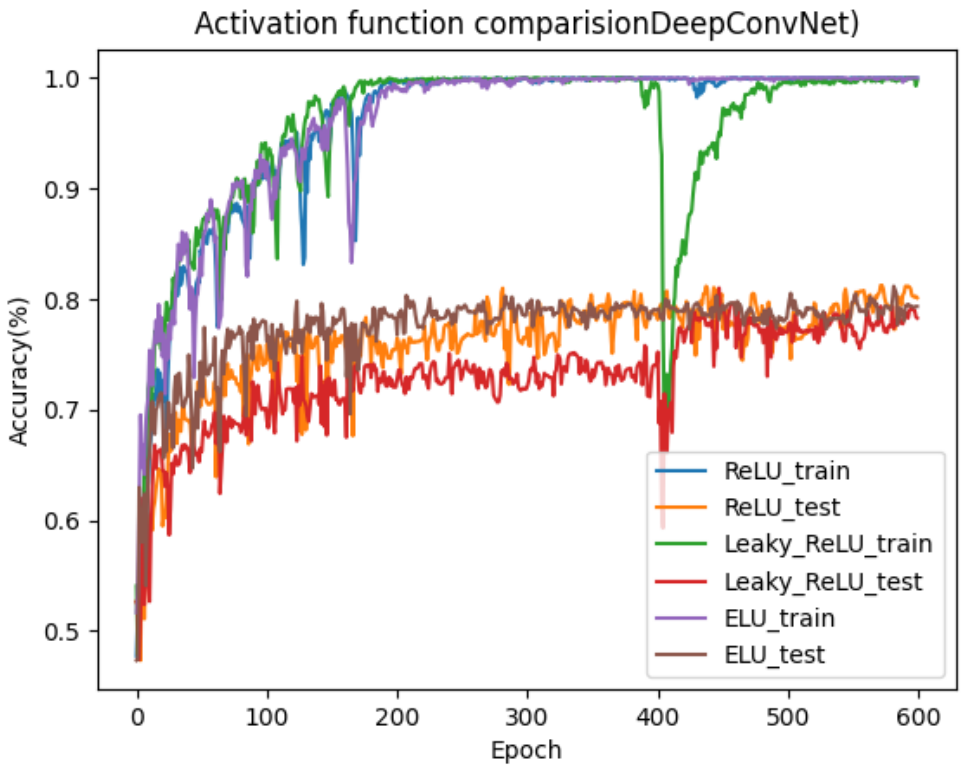
- Data colored in blue indicates that as the learning rate increases, the accuracy also increases.
- More details will be mentioned in Discussion part.

B. Comparison figures

- EEGNet:



· DeepConvNet:



4. Discussion

A. Batch size

- Through the above experiments, we can find that the accuracy increases with batch size within a reasonable range. I think the reasons are as follows:
  - **Increased Training Stability**: Larger batch sizes often lead to more stable training because the gradient estimates are more accurate. This can result in smoother convergence and less oscillation in the loss, which might lead to slightly higher accuracy.
  - **Faster convergence**: Larger batch sizes may lead to faster convergence since each update covers more samples, effectively taking larger steps in the weight space. This allows good solutions to be found more quickly and potentially achieve higher accuracy.
- However, in some cases, increasing the batch size too much might cause some negative effects:
  - **Decreased accuracy**: This could happen due to overfitting or the model getting stuck in a suboptimal region.
  - **Less Computationally Efficiency**: Larger batch sizes result in increased memory usage and computation time, especially on GPU devices. If the batch size becomes too large and exceeds the available memory, the training may crash or fail.

B. Learning rate

- In my code, I use the AdamW optimizer with a CosineAnnealingWarmRestarts scheduler to adjust the learning rate, which means that the learning rate gradually decreases during the training process.
- Through the experiments mentioned above, we can observe that the accuracy improves as the learning rate increases within a sensible range. I assume the reasons for this are as follows:
  - **Faster Convergence**: With a higher learning rate, the model can update its parameters more aggressively, leading to faster

convergence. This rapid convergence might allow the model to reach a good solution in fewer epochs, which can increase accuracy.

- **Exploring diverse solutions**: A larger learning rate can lead to the optimization process exploring a wider range of parameter space, potentially escaping local minima and finding better solutions. This exploration may result in an increase in accuracy.

- However, in some cases, increasing the learning rate too much might cause some negative effects:

  - **Overshooting optimal solutions**: The updates become so significant that the model starts oscillating around the minimum, preventing it from converging to a good solution. As a result, accuracy may decrease.

  - **Divergence**: If the learning rate is too high, the model's parameter updates may become too large, leading to unstable behavior during training. The loss may increase instead of decreasing, and the model may fail to converge. This divergence will result in a decrease in accuracy.